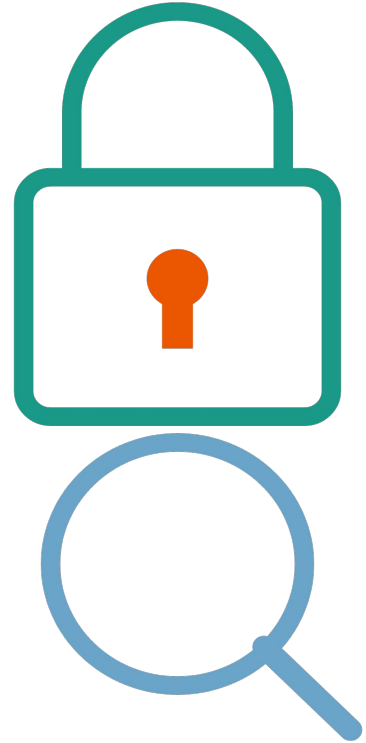


Static & Configuration Analysis

*Session 10 - Reading code and configs without
running anything*





S10 Static & Configuration Analysis



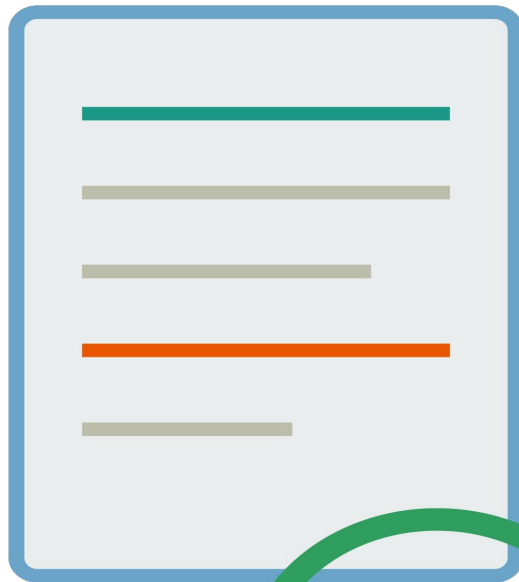
The complement to Session 9: reading code and configuration instead of running tools against a live target.

Session 10 Overview



The most accessible, highest-value evidence you will gather all module

- What SAST finds, misses, and why every finding needs triage (LO2)
- Reading HTTP header gaps, TLS grades, and CORS risk (LO4)
- Mapping a config gap to its property and OWASP category (LO2, LO4)
- Drafting a professional finding from config evidence (LO2, LO4)



Completing the Tools Block

S9 - Dynamic Tools (recap)

Run tools against something

Nmap, ZAP, and Burp Suite Community actively probe a live target. Evidence comes from watching the system respond.

S10 - Static & Config (today)

Read what's already there

SAST reads source code. Config review reads headers, TLS setup, and CORS policy. Nothing is executed or probed - the most accessible evidence source in the module.

Order note: dynamic before static - S9's caution about tool output as evidence, not verdict, carries straight into today.

SAST: Reading Code Without Running It



- **Static Application Security Testing analyses source code or compiled artefacts - without executing the application**
- **Finds: hardcoded credentials, known insecure function calls, missing input validation, insecure dependencies**
- **Layer: Application (extends to System when scanning compiled artefacts and dependencies)**

Analogy

A copy-editor reading a manuscript before it goes to print catches a misused word or a broken reference on the page - but can't tell you whether the plot makes sense once a reader goes through it.

SAST: Finds vs Misses



Finds

Visible as patterns in the code

Hardcoded credentials. Known insecure function calls.
Missing input validation. Insecure dependencies.

Misses

Only visible once it runs

Logic flaws. Runtime behaviour. Context-dependent vulnerabilities - anything that only shows up when the application is actually executing.

The False-Positive Problem



- **SAST tools flag PATTERNS, not confirmed vulnerabilities**
- **Every finding requires human triage before it becomes a report finding**
- **Same discipline as S9's dynamic tools: tool output is evidence, not a verdict - static and dynamic are complementary, each finds what the other misses**

Coursework hook: SAST awareness feeds the DevSecOps discussion in B4.

DevSecOps Placement: Find It Early, Find It Cheap

- 
- **SAST is most valuable inside the CI/CD pipeline - not as a post-deployment activity**
 - **Principle: find security issues as early and as cheaply as possible**
 - **A design choice for B4 (application security engineering). Layer: Application**

Analogy

A spellchecker that flags a typo as you type costs a keystroke. The same typo caught by a proofreader after the book is printed costs a reprint.

Configuration Review as a Discipline



- Reviewing the configuration of a deployed system against a known-good baseline
- Session 6 named OWASP A05: Security Misconfiguration but did not give it a review discipline - this session closes that loop
- Covers: HTTP security headers, TLS configuration grading, CORS policy. Layers: Application, Network

Which Property Is Each Header Protecting?



- **CSP -> Integrity** (stops an unauthorised script from altering page behaviour)
- **HSTS -> Confidentiality + Authenticity** (forces HTTPS, blocks a false endpoint intercepting traffic)
- **X-Frame-Options -> Integrity** (stops the page being framed and manipulated by another site)
- **X-Content-Type-Options -> Integrity** (stops the browser being tricked into misreading content type)
- **Referrer-Policy -> Confidentiality** (stops the URL leaking to a third-party destination)

The Rules Posted at the Door



- **Each header is a rule the browser is told to enforce for this site - unrelated to what happens at the server**
- **A missing header is a rule that was never posted - the browser has no way to enforce a policy it was never given**

Analogy

HTTP security headers are the notices pinned to a venue's front door: no side entrances (HSTS), guest list only (CORS), no photography past this point (CSP). The bouncer only enforces what's posted.

Five Headers, Five Attacks Prevented



Header	Attack Prevented
Content-Security-Policy (CSP)	XSS - restricts which scripts can execute
HTTP Strict Transport Security (HSTS)	Protocol downgrade, MITM - forces HTTPS
X-Frame-Options	Clickjacking - prevents embedding in iframes
X-Content-Type-Options	MIME sniffing attacks
Referrer-Policy	Information leakage via the Referrer header

KEY CONCEPT

A Configuration Gap Is a Property Failure

A missing CSP or HSTS header is not a cosmetic nit-pick. It can be exploited to intercept a session - which breaks Authenticity AND Accountability together: the attacker impersonates a genuine party, and every action from that point on can no longer be traced to who really did it.



From Headers to STRIDE to Your SRS

- CSP gaps map to Tampering (an attacker alters page behaviour via injected script)
- Referrer-Policy / CORS gaps map to Information Disclosure (data leaks to a destination it should never reach)
- Coursework hook: every gap you find here feeds APP-01 or APP-02 in your SRS

Try it: the Security Headers + TLS Grade demo lets you toggle each header on and off and watch the attack it blocks - and the grade - move in real time.

TLS Configuration: A Grade to Read, Not Compute

- 
- **Weak TLS = old protocol versions (TLS 1.0, 1.1), weak cipher suites, missing HSTS, certificate chain issues**
 - **SSL Labs grades a configuration automatically - like CVSS, you read the score, you do not compute it (no maths, consistent with S3)**
 - **Layer: Network**

Analogy

An MOT certificate doesn't ask you to understand the engine - it hands you a pass/fail and a list of what failed. SSL Labs' grade works the same way for a server's TLS setup.

When the Grade Drops



- A server still accepting TLS 1.0 lets an attacker force a protocol downgrade - then intercept traffic that was supposed to be encrypted
- Property broken: Confidentiality (and Authenticity, if the intercepted channel is then impersonated)
- Coursework hook: NET-02 - your SRS requirement for transport security

CORS: Who Else Is Allowed to Ask?

- **Cross-Origin Resource Sharing controls which other websites' scripts may read this site's responses**
- **Access-Control-Allow-Origin: * means ANY website's script may read the response**
- **Layer: Application**

Analogy

A wildcard CORS policy is a guest list that just says 'everyone's invited.' The bouncer stops checking who's actually on it.

What a Wildcard Actually Allows



- Combined with credentials (cookies, auth tokens), a wildcard CORS policy lets a malicious site's script make authenticated requests on the victim's behalf and read the response
- Property broken: Confidentiality - and Integrity if the attacker can also trigger a state-changing action
- Coursework hook: APP-01 / APP-02

The Format You'll Use in the Lab



- Title - one line naming the gap
- Severity - from your CVSS score (reuse the calculator from S8/S9)
- Evidence - the header, grade, or policy you observed
- Property violated - name it explicitly
- Remediation recommendation - the fix, in one sentence

Same discipline as S8 and S9 - reused, not re-taught.

Activity: Grade Three Sites

*securityheaders.com on 3 real-world sites
(at least one from your assigned domain):*

1. Grade each
2. For each missing header, name the specific attack it enables
3. Map each gap to a security property and an OWASP category
4. Identify what a SRS requirement for this control would look like

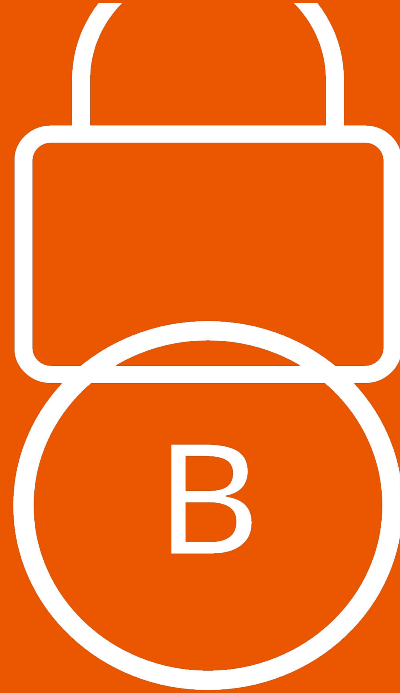


Small Lab: SSL Labs Scan

SSL Labs scan on a provided domain:

1. Interpret the grade report
2. Identify what is reducing the score
3. Draft one finding: title, severity, evidence, property violated, remediation recommendation

Site down? Use the pre-saved grade report instead.



Where This Session Lands in Your Report



- **Configuration review is the most accessible, highest-value evidence source for your report - say this to yourself when you are short on time**
- **Feeds APP-01, APP-02, NET-02 traceability directly**
- **The evidence format practised here carries straight into Part B**
- **SAST awareness supports the DevSecOps discussion in B4**

Questions & Answers



Next: Session 11 - Observability, Compliance & Resilience



Disclaimer & Copyrights



The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I do make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability concerning the content within this presentation.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites. I am not responsible for the content of any linked site or any link contained in a linked site.

Any action you take upon the information presented in this presentation is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.

@ Images and artworks used in this presentation are used for general informational purposes only. It may contain copyrighted material, the use of which may not have been specifically authorised by the copyright owner. This material is available in an effort to explain issues relevant to the topic. This should constitute a 'fair use' of any such copyrighted material.

Last modified: 10 Jul 2026